

Esquema de Datos MapReduce

Contratos de datos del sistema de búsqueda de secuencias genómicas

DistributedProcessing, Documento 03-esquema-datos

Este documento define los contratos y estructuras de datos para las diferentes fases del ciclo de vida de procesamiento distribuido MapReduce dentro del sistema de búsqueda de secuencias genómicas en formato FASTA. Se detallan los formatos del chunk de entrada, la salida intermedia de Map, la entrada de Reduce y el esquema de resultado final consolidado.

1. Fase de entrada: estructura del chunk (InputChunk)

Cada Worker recibe un fragmento del genoma, o chunk, generado por el Master o el servicio de particionado. Para evitar perder patrones en las fronteras de división, cada bloque incluye un margen de solapamiento, u overlap, de $L - 1$ bytes, donde L es la longitud del patrón buscado.

1.1. Definición de campos

Campo	Tipo	Descripción
chunk_id	string	Identificador único del bloque, por ejemplo chr1_chunk_004.
source_file	string	Nombre o identificador del archivo FASTA de origen.
target_pattern	string	Secuencia de ADN o patrón a buscar en este trabajo, por ejemplo ATGCGATC.
start_offset	uint64	Posición global absoluta donde inicia el bloque propio en el genoma.
end_offset	uint64	Posición global absoluta donde termina el bloque propio.
overlap_size	uint32	Número de bytes adicionales leídos del siguiente chunk ($L - 1$).
sequence_data	string	Cadena de nucleótidos: bloque propio más margen de overlap.

1.2. Aclaraciones de campos

El campo chunk_id es indispensable para la reasignación de tareas en caso de que un Worker falle. El campo source_file permite que el sistema procese genomas divididos en múltiples archivos o ejecute trabajos sobre diferentes secuencias de forma concurrente.

El campo start_offset indica la posición absoluta, en pares de bases, donde empieza el bloque dentro del genoma completo, y actúa como límite inferior para que el Worker descarte coincidencias que inicien antes de ese punto. El campo end_offset representa el límite superior del bloque propio asignado al Worker, y sirve para evitar el conteo duplicado: aunque el Worker analice datos más allá de este punto debido al solapamiento, solo emitirá patrones cuyo inicio esté estrictamente dentro del intervalo entre start_offset y end_offset.

El campo `overlap_size` cuantifica exactamente cuántos bytes o nucleótidos extra contiene el bloque al final, equivalentes a $L - 1$, y permite al Worker validar la integridad de la ventana de lectura en los límites del chunk. El campo `sequence_data` constituye la carga útil de procesamiento, o payload, del chunk.

2. Fase intermedia: pares clave-valor (MapOutput)

La función Map analiza el `sequence_data` y genera tuplas compuestas por una clave, el patrón, y un objeto estructurado `MatchInfo` con el detalle de la posición localizada.

2.1. Estructura del objeto MatchInfo (Value)

Campo	Tipo	Descripción
<code>source_file</code>	<code>string</code>	Archivo o secuencia de referencia donde se encontró la coincidencia.
<code>global_offset</code>	<code>uint64</code>	Posición absoluta del primer nucleótido del patrón en el genoma.
<code>line</code>	<code>uint32</code>	Número de línea en el archivo de origen.
<code>column</code>	<code>uint32</code>	Columna o posición relativa local.

2.2. Representación del par intermedio (Key, Value)

La clave es `pattern`, de tipo `string`, correspondiente a la secuencia buscada o al identificador del motivo. El valor es `match`, de tipo `MatchInfo`, correspondiente a los metadatos de la ocurrencia.

Formato del par:

```
Key:  "ATGCGATC"
Value: MatchInfo {
    source_file:  "homo_sapiens_chrl.fasta",
    global_offset: 1450234,
    line:         18128,
    column:       34
}
```

2.3. Aclaraciones de campos

Establecer la clave como `string` permite que el sistema admita búsquedas simultáneas de múltiples motivos; durante la fase de Shuffle, el clúster agrupará automáticamente todas las ocurrencias bajo su patrón correspondiente.

El valor se estructura como un objeto compuesto, `MatchInfo`, en lugar de un número simple, para transportar el contexto biológico completo de la aparición. El campo `source_file` permite conocer con precisión en qué cromosoma o archivo se produjo el hallazgo cuando se analizan genomas fragmentados. El campo `global_offset` es la coordenada absoluta global del inicio del patrón en el genoma, y constituye el dato principal para el ordenamiento posterior y el mapeo genómico. El campo `line` corresponde al número de línea dentro del archivo FASTA original, y facilita la verificación manual e inspección rápida del archivo fuente. El campo `column` indica la posición horizontal dentro de la línea del archivo de origen, completando las coordenadas exactas de localización.

3. Fase de reducción: entrada de Reduce (ReduceInput)

El proceso de Shuffle y Partitioning agrupa todos los valores intermedios emitidos por los Workers asociados a una misma clave, antes de suministrarlos a la función reductora.

3.1. Definición

La clave es `pattern`, de tipo `string`. Los valores corresponden a `list<MatchInfo>`, una colección iterable de todas las ocurrencias emitidas por los diferentes Workers para ese patrón.

3.2. Aclaraciones de campos

La clave `pattern` identifica el grupo de datos que entra a reducirse, y garantiza que cada función Reduce trabaje de forma aislada sobre un único motivo o subsecuencia. La colección `list<MatchInfo>` reúne todos los objetos de coincidencia generados por la fase Map a lo largo del clúster; al recibir la lista completa en memoria o en stream, el Reducer cuenta con los elementos necesarios tanto para contar el total de frecuencias como para ejecutar un algoritmo de ordenamiento sobre las posiciones encontradas.

4. Fase de salida: resultado final (FinalResult)

El resultado que produce la función Reduce consolida la frecuencia total y el inventario ordenado de ubicaciones encontradas.

4.1. Definición de campos

Campo	Tipo	Descripción
<code>job_id</code>	<code>string</code>	Identificador del trabajo de procesamiento.
<code>pattern</code>	<code>string</code>	Patrón consultado.
<code>total_matches</code>	<code>uint64</code>	Número total acumulado de coincidencias encontradas.
<code>matches</code>	<code>list<MatchInfo></code>	Lista de todas las coincidencias ordenadas por <code>global_offset</code> .
<code>execution_time_ms</code>	<code>uint64</code>	Tiempo total de cómputo transcurrido, para métricas.

4.2. Aclaraciones de campos

El campo `job_id` identifica de forma global el trabajo de búsqueda ejecutado, y permite indexar, almacenar y consultar los resultados de forma unívoca en la base de datos de resultados. El campo `pattern` confirma la secuencia de ADN asociada a la respuesta devuelta al cliente o usuario solicitante.

El campo `total_matches` es el conteo acumulado de ocurrencias; proveerlo como campo explícito evita que las aplicaciones cliente tengan que recorrer o medir la lista de resultados únicamente para conocer cuántas veces apareció el patrón. El campo `matches` reúne el arreglo final consolidado con todos los detalles de ubicación, entregado ordenado cronológicamente por `global_offset`, listo para graficar o generar reportes bioinformáticos. El campo `execution_time_ms` indica el tiempo que tomó la ejecución del procesamiento, dato vital para las métricas de rendimiento del clúster, los benchmarks y la evaluación de throughput del sistema distribuido.

5. Especificación de contratos para gRPC / Protocol Buffers

Dado que la arquitectura se implementa sobre Go y gRPC, los contratos conceptuales anteriores se mapean a la siguiente definición formal en Protocol Buffers v3.

```
syntax = "proto3";

package mapreduce;

option go_package = "./proto";

// Representa el bloque asignado al Worker
message InputChunk {
    string chunk_id = 1;
    string source_file = 2;
    string target_pattern = 3;
    uint64 start_offset = 4;
    uint64 end_offset = 5;
    uint32 overlap_size = 6;
    string sequence_data = 7;
}

// Representa una coincidencia puntual
message MatchInfo {
    string source_file = 1;
    uint64 global_offset = 2;
    uint32 line = 3;
    uint32 column = 4;
}

// Mensaje emitido por el Worker tras procesar el chunk
message MapResponse {
    string chunk_id = 1;
    string pattern = 2;
    repeated MatchInfo matches = 3;
}

// Estructura consolidada entregada por el Reduce y persistida
message FinalResult {
    string job_id = 1;
    string pattern = 2;
    uint64 total_matches = 3;
    repeated MatchInfo matches = 4;
    uint64 execution_time_ms = 5;
}
```

5.1. Aclaraciones de campos

La directiva `repeated` de Protocol Buffers serializa una lista dinámica de forma binaria eficiente, eliminando delimitadores de texto como corchetes o comas de JSON y reduciendo el tráfico de red en el clúster. En cuanto a compatibilidad de tipos escalares, los enteros grandes se mapearon a `uint64`, para offsets y métricas, y los índices locales a `uint32`, para filas y columnas, optimizando el empaquetado binario mediante codificación varint de Protocol Buffers.